`const` means "I won't change it". `constexpr` means "the compiler knows it".

```
const int a = std::rand();
constexpr int b = 5 * 2;
```

Both are read-only. Only one of them is known before the program runs.

swipe →

`const` can wait for run time

```
const int a = std::rand();    // ok
constexpr int b = std::rand(); // error
```

`const` only promises not to change **after** initialization.

The value can come from input, a file, a random number.

`constexpr` demands a value the compiler can work out itself.

Why it matters

```
constexpr int n = 4;
std::array<int, n> a;       // ok

const int m = std::rand();
std::array<int, m> b;       // error
```

Array sizes, template arguments and `static_assert` need a compile-time value.

A `const int` with a literal initializer also works there, which hides the difference. `constexpr` makes it a guarantee.

constexpr functions

```
constexpr int square(int x) {
    return x * x;
}

static_assert(square(4) == 16);   // compile time
int r = square(std::rand());      // run time
```

The same function runs in **both** worlds.

Given constants, it can run during compilation. Given run-time values, it runs normally.

Side by side

	CONST	CONSTEXPR
read-only	**yes**	**yes**
initializer	may be run time	must be compile time
array size, template argument	only if constant	**always**

All of these were checked with the compiler.

Which one?

YOU HAVE	WRITE
a fixed number, known now	`constexpr`
a value set once at run time	`const`
a function usable in both	`constexpr`

Do you still use `#define` for constants?